MODULE 2

COOPERATING PROCESS

- The processes which execute concurrently and affect or get affected by other processes executing in the system are called cooperating processes.
- The processes can affect or get affected by other processes by sharing data or by sharing other resources in the system.
- Thus any process which shares data with other process is a cooperating process.
- The need of such processes is obvious. They will be required to speed up the computation, sharing of information, user convenience etc.
  Breaking up of one task into subtasks such that each of them can run in parallel and thus give faster computation results may be one of the requirements.
- In a multi programming environment, processes executing concurrently.
- Concurrent execution of cooperating process requires mechanisms that allow processes to communicate with one another and to synchronize their actions.
- Examples of cooperating processes are transaction processes in airline reservation system. They share a common database, update and read same data on a sharing basis.

RACE CONDITION

Consider two cooperating processes, sharing variables A and B and having the following set of instructions in each of them:

Process 1	**Process 2**	**Concurrent access**
A=1 **A=B+1** **B=B+1**	**B=2** **B=B*2**	**Does not matter**

- Suppose our intention is to get A as 3 and B as 6after the execution of both the processes. The interleaving of these instructions should be done in order to avoid race condition.
- If the order of execution is like:

  A=1
  B=2
  A= B+1
  B=B+1
  B=B*2

  whereas if the order of execution is like:

  A=1
  B=2
  B=B*2
  A= B+1
  B=B+1 A will contain 5 and B will contain 5 which is not desired.
- Thus the output of the interleaved execution depends on the particular order in which the access takes place.
- If several processes access and manipulate the same data concurrently, the outcome of the execution depends on the Particular order in which the access takes place. This is called **Race condition.**
- To solve this problem, shared variables A and B should not be updated simultaneously by process 1 and 2.
- Only one process at a time should manipulate the shared variable.
- For that purpose we can use **synchronization mechanism.**

CRITICAL SECTION PROBLEM

- Consider a system consisting of n processes {P0, P1, ..., Pn−1}.
- Each process has a segment of code**, called a critical section** in which the process may be changing common variables, updating a table, writing a file, and so on.
- The important feature of the system is that, when one process is executing in its critical section, no other process is allowed to execute in its critical section.

- That is, no two processes are executing in their critical sections at the same time.
- The critical-section problem is to design a protocol that the processes can use to cooperate.
- Each process must request permission to enter its critical section. The section of code implementing this request is the entry section.
- The critical section may be followed by an exit section.
- The remaining code is the remainder section.
- Thus the critical section of a process should not be executed concurrently with the critical section of another process.
- This should be ensuring by the synchronization mechanism.
- The general structure of a typical process Pi is shown in Figure:

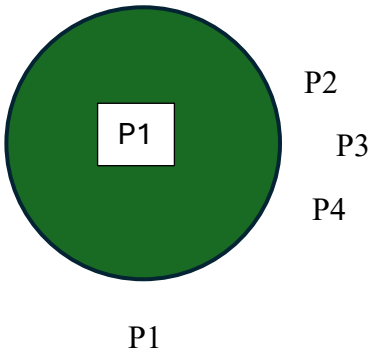
```
do {
        entry section
            critical section
        exit section
            remainder section
} while (true);
```

A solution to the critical-section problem must satisfy the following three requirements:

- **Mutual exclusion:** If process Pi is executing in its critical section, then no other processes can be executing in their critical sections. (no two processes will simultaneously be inside their critical section)
- **Progress:** If no process is executing in its critical section and some processes wish to enter their critical sections, then only those processes that are not executing in their remainder

sections can participate in deciding which will enter its critical section next, and this selection cannot be postponed indefinitely.

- **Bounded waiting:** There exists a bound, or limit, on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.



TestAndSet instruction

- **Mutual exclusion is a property of process synchronization which ensures that if one process is executing in its critical section, no other process is allowed to execute in its critical section. The Test-And-Set instruction is a hardware-based solution used to implement "Spin Locks" to achieve this.**
- The TestAndSet instruction is an **atomic** hardware instruction. Atomicity ensures that the operation (testing the value and setting it) happens as a single, uninterruptible unit.
- **The logic of the instruction (Conceptual):**

```
boolean TestAndSet (boolean *target) {
 boolean rv = *target;          // 1. Store the old value
 *target = TRUE;                // 2. Set the value to true
 return rv;                     // 3. Return the old value
}
```

To solve the mutual exclusion problem, a shared boolean variable (often called lock) is used. It is initialized to false (or 0).

The Algorithm

A process wishing to enter its critical section executes the following:

1. **Entry Section:** The process enters a while loop, calling TestAndSet(&lock).

- If the lock is false, TestAndSet returns false (breaking the loop) and simultaneously sets the lock to true.
- If the lock is already true, TestAndSet returns true, and the process continues to "spin" in the loop.

2. **Critical Section:** The process performs its task.
3. **Exit Section:** The process sets lock = false, allowing waiting processes to enter.

Evaluation of the Solution

For a high-mark response, you must explain how it meets the criteria for a synchronization solution:

- **Mutual Exclusion:** Guaranteed. Only one process can receive a false return value from the atomic instruction at a time.
- **Progress:** If no process is in the critical section, a process that executes TestAndSet will enter immediately.
- **Bounded Waiting:** Standard Test-And-Set does **not** inherently guarantee bounded waiting; a process could theoretically spin forever if other processes keep "winning" the lock.

MODULE 1

HARDWARE PROTECTION

- In early computers, programmers had complete control over the system. As operating systems developed, this control was given to the operating system.
- To improve system utilization, the operating system began to share system resources among several programs simultaneously.
- Multiprogramming put several programs in memory at the same time.
- Errors can occur in a multiprogramming system, where one erroneous program might modify the program or data of another program, or even the resident monitor itself. Many programming errors are detected by the hardware. These errors are normally handled by the operating system.
- If a user program fails in some way — such as by making an attempt either to execute an illegal instruction, or to access memory that is not in the users address space — then the hardware will trap to the operating system.
- The trap transfers control through the interrupt vector to the operating system, just like an interrupt.
- Whenever a program error occurs, the operating system must abnormally terminate the program.

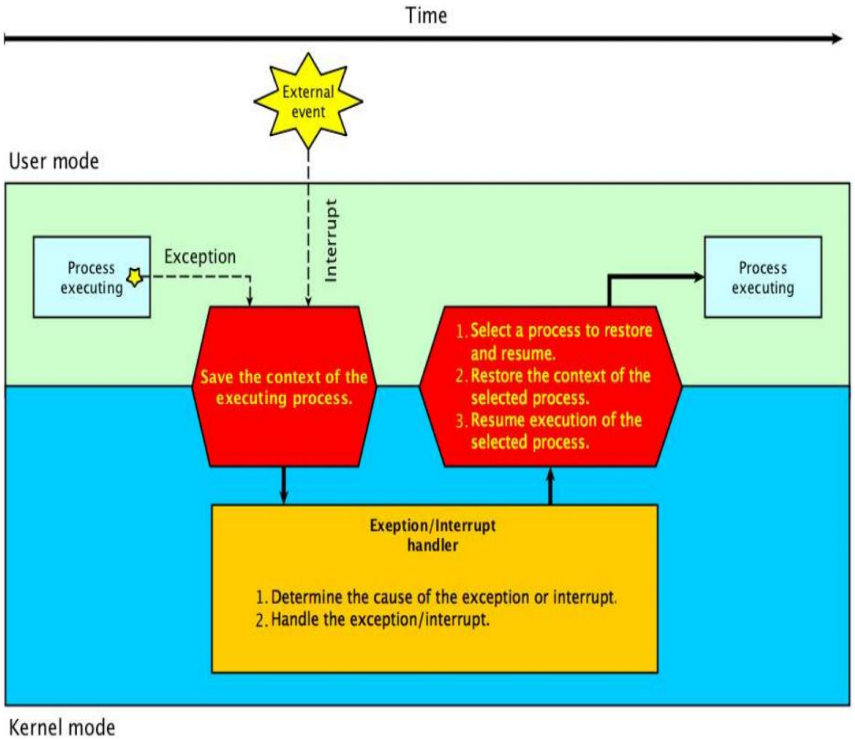
- Dual-Mode Operation

• To ensure proper operation, we must protect the operating system and all other programs and their data from any malfunctioning program. Protection is needed for any shared resource.

- The two separate modes of operations are:

User mode

- Monitor mode (also called Kernel mode, supervisor mode, system mode, or privileged mode).
- A bit, called the mode bit, is added to the hardware of the computer to indicate the current mode: monitor (0) or user (1).
- At system boot time, the hardware starts in monitor mode. The operating system is then loaded, and starts user processes in user mode.
- Whenever a trap or interrupt occurs, the hardware switches from user mode to monitor mode (that is, changes the state of the mode bit to 0).
- Thus, whenever the operating system gains control of the computer, it is in monitor mode.
- The system always switches to user mode (by setting the mode bit to 1) before passing control to a user program.
- The dual mode of operation provides us with the means for protecting the operating system from errant users.
- We accomplish this protection by designating some of the machine instructions that may cause harm as privileged instructions.
- The hardware allows privileged instructions to be executed only in monitor mode.
- If an attempt is made to execute a privileged instruction in user mode, the hardware does not execute the instruction, but rather treats the instruction as illegal and traps it to the operating system.


Time
External event
User mode
Interrupt
Process executing
Exception
Process executing
Save the context of the executing process.
1. Select a process to restore and resume.
2. Restore the context of the selected process.
3. Resume execution of the selected process.
Exeption/Interrupt handler
1. Determine the cause of the exception or interrupt.
2. Handle the exception/interrupt.
Kernel mode